

Activity V : Public Key Infrastructure

1. From the two given openssl commands, what is the difference?

Note: If your operating system does not show any error in the first command, try **openssl s_client -connect twitter.com:443 -CApath /dev/null**. If the results are still the same, your system is not reliable. You may ignore this exercise.

(Modern versions of Mac OS X will always read CA from keychains. There is no intuitive way to turn it off.)

- The terminal gave the same result for both cases.

```

Activity_V-Public_Key_Infrastructure — openssl s_client -connect twitter.com:443 -CApath /tmp

shriria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl s_client -connect twitter.com:443 -CApath /tmp
CONNECTED(00000006)
depth=2 C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root CA
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert TLS Hybrid ECC SHA384 2020 CA1
verify return:1
depth=0 C = US, ST = California, L = San Francisco, O = "Twitter, Inc.", CN = twitter.com
verify return:1
-----
Certificate chain
 0 s:/C=US/ST=California/L=San Francisco/O=Twitter, Inc./CN=twitter.com
  i:/C=US/O=DigiCert Inc/CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
 1 s:/C=US/O=DigiCert Inc/CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
  i:/C=US/O=DigiCert Inc/OU=www.digicert.com/CN=DigiCert Global Root CA
-----
Server certificate
-----BEGIN CERTIFICATE-----
MIIFcTCCBPagAwIBAgIQBhwg1EWwtJeH2bE714udjAKBgqgghkjoPPQDQAZBWMQsw
CQYDVQQGEwJVUzEVMBMGA1UEChMRRGlnaUNlcnQgSW5jMTAwLgYDVQQDEyEaWdp
Q2VydCBUTFmGSHlicmlkIEVDQyBTSEeZODQGMjAyMCBBDQTEwHhcNMjIwMjA5
MDAwHhcnMjYwMmJpAxMjMTOTU5WjBoMQswCQYDVQQGEwJVUzETMBEGA1UECBMkQ2Fs
aWZvcn5pY2EwMBQGA1UEBwMNU2FuIEZyYW5jaXNjbzEWBQGA1UEChMNVHdpdHRl
ciwgSW5jbjEUMBIGA1UEAxiMLdHdpdHRlci5jb2wWATBgcqhkiOQPQIBBgqghkjo
PQMBBwNCAAR/wMaDpejym7271zu2VxzjiMzmDGr1p7qtgULiHcxrWTSJOxGFvWg
xmcS/WJpnczn2gaxR8QjrkgFGcRjb+5o4IdkCCA44wHwYDVR0jBBgwFoAUCrwi
KReMpt1teg7OM8cus+37w3owHqYDVR00BBYEFAFzO8rUoQLYL35jIsTyH22LQw0
MCcGA1UdEQQgMB6CC3R3aXR0ZXIuY29tg93d3cudHdpdHRlci5jb2wWdGyDVR0P
AQH/BAQDAgeAMB0GA1UdJQQWMBQGCCsGAQUFBwMBBggrBgEFBQcDAjCBmwyDVR0f
BIGTMIQGMEagRKBCbkBodHRwOi8vY3JsMy5kaWdpY2VydC5jb20vRGlnaUNlcnRU
TFNlIeWJyaWRFRQ0NTSEeZODQyMDIwQ0ExLTEuY3JsMEagRKBCbkBodHRwOi8vY3Js
NC5kaWdpY2VydC5jb20vRGlnaUNlcnRUTFNlIeWJyaWRFRQ0NTSEeZODQyMDIwQ0Ex
LTEuY3JsMDQ4GA1UdJTAQMDUwMwyGZ4EMAQICMCKwJwYTKwYBBQUHAQEEG2h0dHA6
Ly93d3cuZGlnaWNLcnQuY29tL0NQUZCBhQYIKwYBBQUHAQEETB3MCGCCCsGAQUF
BzABhhnHdHRwOi8vbnZlc3k5kaWdpY2VydC5jb20wT2wYIKwYBBQUHMAKG2h0dHA6
Ly9jYWNlcnRzLmRpZ21jZXJ0LmNvbS9EaWdpQ2VydFRMU0h5NjZpZEVkdQlNlQTM4
NDIwMjBDQTEtMS5jcnQwDAYDVR0TAQH/BAIwADCCAX4GcisGAQQB1nkCBAIEggFu
-----

```

```
Activity_V-Public_Key_Infrastructure — openssl s_client -connect twitter....

---
No client certificate CA names sent
Server Temp Key: ECDH, X25519, 253 bits
---
SSL handshake has read 2918 bytes and written 289 bytes
---
New, TLSv1/SSLv3, Cipher is ECDHE-ECDSA-AES128-GCM-SHA256
Server public key is 256 bit
Secure Renegotiation IS supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol : TLSv1.2
    Cipher    : ECDHE-ECDSA-AES128-GCM-SHA256
    Session-ID: 8BA78BD7EEA286F53E15875B7F29EA339C1E4DDF9A2EDD5CA84C7C5D52A06F1F
    Session-ID-ctx:
    Master-Key: 413BEA1A2AB5ABB1486662A8BCA68CF673349A6D520E07A36E7E822EC6E4F737
    34EE32D41A2927836EF0A90C016CB923
    TLS session ticket lifetime hint: 7200 (seconds)
    TLS session ticket:
    0000 - 71 a6 fa 04 92 13 3b d4-dd b2 20 56 b2 09 e1 ab   q.....;... V....
    0010 - 8b fb 6a 2a 95 74 c0 35-2a 6c 5c 83 b8 6d 5c f5   ..j*.t.5*1\..m\..
    0020 - f1 94 15 9e 9e ea a0 ed-3c 8c ee af ba 44 51 d0   .....<....DQ.
    0030 - 0e 66 0a 70 2d 9f 2c 3c-0f ac 33 16 80 c2 2e 5b   .f.p-.,<...3....[
    0040 - f2 a7 22 49 a7 b1 87 a9-f8 8a e0 11 a4 94 f7 93   .."I.....
    0050 - c2 ce d5 97 ff 06 97 d0-51 cb 57 bf ed 2a c4 0a   .....Q.W.*..
    0060 - ce ba 98 ab 11 d9 08 38-da 40 5e 1e 46 5f 26 21   .....8.@^..F_&!
    0070 - b4 70 2f aa e3 f6 0c a3-26 66 17 1c ff cb b2 05   .p/.....&f.....
    0080 - ad b3 a4 93 f0 92 db 68-de fb b4 75 5e 4c bc a5   .....h....u^L..
    0090 - 56 ff 3d 8a 44 62 bd df-c0 71 f4 03 c9 fa b0 ea   V.=.Db...q.....
    00a0 - 29 c3 16 4b 21 59 8e 35-77 fd 34 70 b1 74 89 ad   )..K!Y.5w.4p.t..
    00b0 - 74 ad 1f eb 49 a4 af b4-a1 f0 6b a8 7a ff 99 1e   t...I.....k.z...

    Start Time: 1663039034
    Timeout    : 7200 (sec)
    Verify return code: 0 (ok)
---
```

2. What does the error (**verify error**) in the first command mean? Please explain.

- From my device (MacOS), there's no errors occurred. But there would be such an error message as "Unable to get local issuer certificate" on Windows/Linux OS, because of no files existing at the argument 'CApath' we have inputted into the command.

- Copy the server certificate (beginning with -----BEGIN CERTIFICATE----- and ending with -----END CERTIFICATE-----) and store it as `twitter_com.cert`. Use the command `openssl x509 -in twitter_com.cert -text` to show a text representation of the certificate content. Briefly explain what is stored in an X.509 certificate (i.e., data in each field).

```
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl x509 -in twitter_com.cert -text
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      04:7c:20:d4:45:96:b4:97:87:65:bd:84:ef:5e:2e:76
    Signature Algorithm: ecdsa-with-SHA384
    Issuer: C=US, O=DigiCert Inc, CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
    Validity
      Not Before: Feb  2 00:00:00 2022 GMT
      Not After : Feb  1 23:59:59 2023 GMT
    Subject: C=US, ST=California, L=San Francisco, O=Twitter, Inc., CN=twitter.com
    Subject Public Key Info:
      Public Key Algorithm: id-ecPublicKey
      Public-Key: (256 bit)
      pub:
        04:7f:c0:c6:83:a5:e8:f2:9b:bd:bb:97:3b:b6:bc:
        4c:73:8e:23:33:98:31:ab:96:9e:ea:b6:05:0b:88:
        77:2b:c5:64:d2:24:ec:46:16:f5:a0:c6:67:12:fd:
        62:69:9d:cc:e7:87:68:1a:c5:1f:10:8e:b9:20:14:
        67:11:25:bf:b9
      ASN1 OID: prime256v1
      NIST CURVE: P-256
    X509v3 extensions:
      X509v3 Authority Key Identifier:
        keyid:0A:BC:08:29:17:8C:A5:39:6D:7A:0E:CE:33:C7:2E:B3:ED:FB:C3:7A
      X509v3 Subject Key Identifier:
        01:59:A3:CA:D4:41:E2:D8:40:BD:F9:8C:8B:13:C8:76:76:2D:0C:34
      X509v3 Subject Alternative Name:
        DNS:twitter.com, DNS:www.twitter.com
      X509v3 Key Usage: critical
        Digital Signature
      X509v3 Extended Key Usage:
        TLS Web Server Authentication, TLS Web Client Authentication
      X509v3 CRL Distribution Points:

    Full Name:
      URI:http://crl3.digicert.com/DigiCertTLShybridECCSHA3842020CA1-1.crl
```

```
Activity_V-Public_Key_Infrastructure -- -zsh -- 108x39

X509v3 CRL Distribution Points:

  Full Name:
    URI:http://crl3.digicert.com/DigiCertTLShybridECCSHA3842020CA1-1.crl

  Full Name:
    URI:http://crl4.digicert.com/DigiCertTLShybridECCSHA3842020CA1-1.crl

X509v3 Certificate Policies:
  Policy: 2.23.140.1.2.2
  CPS: http://www.digicert.com/CPS

Authority Information Access:
  OCSP - URI:http://ocsp.digicert.com
  CA Issuers - URI:http://cacerts.digicert.com/DigiCertTLShybridECCSHA3842020CA1-1.crt

X509v3 Basic Constraints: critical
  CA:FALSE
  1.3.6.1.4.1.11129.2.4.2:
    ...j.h.w....|....=.>.j.g)]...$.4.....~.....H0F!...8...c0I9.,LU..../.c.....x
    !..#.1..;...@B....W..r.U..@ .....v.5.....lW...LmB.H.S.N.?w...!...j-.*F..*Q&=Xwa..q...l..Q.&...+..u..sw...P.
    c.....Jy-g.....y6.....~.....F0D. r.:g..YuQ.N+e.....".....F....dH7. 4...~I>....C...a
    ...!S...
  Signature Algorithm: ecdsa-with-SHA384
    30:66:02:31:00:f8:7e:04:ca:41:96:06:08:a0:b6:69:2c:31:
    d2:2a:f4:6b:fa:45:5e:25:1b:6c:29:fe:3c:b0:c3:9e:96:83:
    47:f0:6e:e0:8d:3c:9f:07:7e:39:72:af:d2:ad:c2:67:64:02:
    31:00:a5:40:f0:aa:9b:a3:b8:07:70:44:d4:de:5b:e2:c6:17:
    e0:cd:a4:b2:4f:e8:95:90:63:c4:d9:1d:5e:7c:79:7f:cf:57:
    8b:e9:15:6d:e2:40:cb:c7:04:68:41:fa:59:39
-----BEGIN CERTIFICATE-----
MIIFcTCCBPagAwIBAgIQBHwg1EWwtJeHZb2E714udjAKBggqhkJOPQQDAzBWMQsw
CQYDVQQGEwJVUzEVMBMGA1UEChMMRG1naUNlcnQgSW5jMTAwLgYDVQQDEydEaWdp
```

- The command displayed an information of the certification from the given certificate, which stored content fields as followings:
 - Version: Version of the certification
 - Serial Number: a unique key for this website
 - Issuer: Certificate Authority whose certificate and private key are stored inside the cluster
 - Validity: Valid date/Expired date of the certification
 - Subject: Owner of this certificate
 - pub: Public key
 - extensions: Certificate Extensions usage

4. From the information in exercise 3, is there an intermediate certificate? If yes, what purpose does it serve?

Hint: Look for an issuer and download the intermediate certificate. You may use the command **openssl x509 -inform der -in intermediate.cert -text** to show the details of the intermediate certificate. (Note that the -inform der is for reading the DER file. The default file format for x509 is the PEM file.)

Authority Information Access:

OCSP – URI:http://ocsp.digicert.com

CA Issuers – URI:http://cacerts.digicert.com/DigiCertTLShybridECCSHA3842020CA1-1.crt

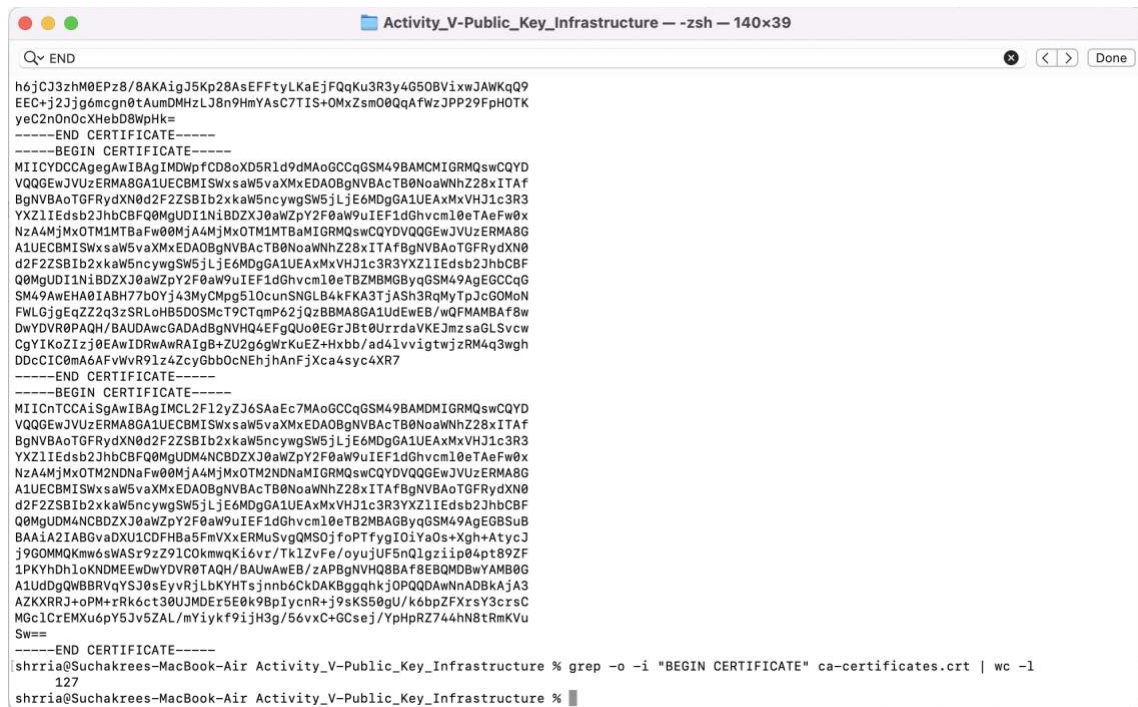
 TLS/SSL PKI IoT Solutions About Support    	
DigiCert TLS RSA SHA256 2020 CA1 Download PEM Download DER/CRT	Issuer: DigiCert Global Root CA Valid until: 13/Apr/2031 Serial #: 06D8D9D4D5584346F68A2FA754227EC4 SHA1 Fingerprint: 1C58A3A8518E8759BF075876B750D4F2DF264FCDD SHA256 Fingerprint: 52274C57CE4D EE3849DB7A7F7F08C040F771898838E88725A86F8443D182FE14
DigiCert Global G2 TLS RSA SHA256 2020 CA1 Download PEM Download DER/CRT	Issuer: DigiCert Global Root G2 Valid until: 23/Sep/2030 Serial #: 085F94C024857648CC144F534D323428 SHA1 Fingerprint: 1D7322841ED99FDD685118AB786C8E26ED831838 SHA256 Fingerprint: 1F8E89E9A8ED66CC583833ED683129764B622639D5B163F600E1C79120BF3EED
DigiCert TLS Hybrid ECC SHA384 2020 CA1 Download PEM Download DER/CRT	Issuer: DigiCert Global Root CA Valid until: 13/Apr/2031 Serial #: 07F2F35C87A877AF7A EF E9479935258D SHA1 Fingerprint: AE C1 3C D0 5E A6 A3 99 8A EC 14 AC 33 1A D9 68 ED 8B 77 DF SHA256 Fingerprint: F7A9A182FD964A3F2670BD66BD561FB7C55D3AA9AB8391E7E169702DBA3DBCF
DigiCert Trusted G4 TLS RSA SHA384 2020 CA1 Download PEM Download DER/CRT	Issuer: DigiCert Trusted Root G4 Valid until: 23/Sep/2030 Serial #: 01052F064A975E F1 F50E9161F5362A8 SHA1 Fingerprint: 9F90F8C866480959C29D9AC9A78AEAB8E26D88EB SHA256 Fingerprint: 4F35C6B9C100905F258801C0A20EBE335BCEE F249CE9FA1E11ED62ED66FCD1
DigiCert Global G3 TLS ECC SHA384 2020 CA1 Download PEM Download DER/CRT	Issuer: DigiCert Global Root G3 Valid until: 29/Mar/2031 Serial #: 0C2254181D60FDFA66E264E3C17A48BC SHA1 Fingerprint: C3BD4F375F3F41A5685A79B9C4A9ADADCC4B30D4 SHA256 Fingerprint: AB6CFACB8D594238D7F6D346E27A294B76E2F215627091A555C7388B1A8B798A

5. Is there an intermediate CA, i.e. is there more than one organization involved in the certification? Say why you think so.

```
depth=2 C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root CA
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert TLS Hybrid ECC SHA384 2020 CA1
verify return:1
depth=0 C = US, ST = California, L = San Francisco, O = "Twitter, Inc.", CN = twitter.com
verify return:1
---
Certificate chain
 0 s:/C=US/ST=California/L=San Francisco/O=Twitter, Inc./CN=twitter.com
  i:/C=US/O=DigiCert Inc/CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
 1 s:/C=US/O=DigiCert Inc/CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
  i:/C=US/O=DigiCert Inc/OU=www.digicert.com/CN=DigiCert Global Root CA
---
```

- From the image, we found that there are 2 CAs found in the Certificate chain. The deepest certification was signed by the DigiCert Global Root CA for the intermediate CA to serve leaf nodes in large number.
 - From this exercise, we found that DigiCert is only one organization serving for the subject "twitter".
 - On the internet, there are a lot of CA organizations from many countries around the world who act as a trusted third party to store, sign, and issue certificates for public-key owners.
6. What is the role of ca-certificates.crt?
- A system file that contains certificates from well-known and trustworthy CAs. When the device sends request to a website and receives a public key of its CA, the file shall be accessed to check an existence of the CA's certificate to authenticate the subject.

7. Explore the ca-certificates.crt. How many certificates are in there? Give the command/method you have used to count.



```
Activity_V-Public_Key_Infrastructure -- zsh -- 140x39
Q~ END
h6jCJ3zhM0EPz8/8AKaigJ5Kp28AsEFFtyLKAEjFQqKu3R3y4G50BVixwJAWKqQ9
EEC+j2Jjg6mcgn0tAumDMHzLJ8n9HmYAsC7TIS+OMXzm00QqAFwzJPP29FpHOTK
yeC2n0OcXHebD8WpHk=
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIICYDCCAgewAwIBAgIMDwPfcD8oXD5R1d9dMAoGCCqGSM49BAMCMIGRMQswCQYD
VQGGewJVUzERMA8GA1UECBMISWxsaw5vaXNxEADA0BgNVBACTB0NoaWZhZ28xITAf
BgNVBAoTGFRydXN0d2F2ZSBib2xkaW5ncywgSW5jLjE6MDgGA1UEAxMVHJ1c3R3
YXZlIEdsb2JhbCBFQ0MgUDU1NiBDZXJ0aWZpY2F0aW9uIEF1dGhvcml0eTAeFw0x
NzA4MjMxOTM1MTBhFw00MjA4MjMxOTM1MTBhMIGRMQswCQYDVQGEwJVUzERMA8G
A1UECBMISWxsaw5vaXNxEADA0BgNVBACTB0NoaWZhZ28xITAfBgNVBAoTGFRydXN0
d2F2ZSBib2xkaW5ncywgSW5jLjE6MDgGA1UEAxMVHJ1c3R3YXZlIEdsb2JhbCBF
Q0MgUDU1NiBDZXJ0aWZpY2F0aW9uIEF1dGhvcml0eTBZMBMGByqGSM49AgEGCCqG
SM49AwEHA0IABH77b0Yj43MyCMpg510cunSGLB4kFKA3TjASH3RqMyTpJcG0MoN
FWLgJgEqZ2q3zSRLOhB5DOSMT9CTqmp62JqzBBMA8GA1UdEwEB/wQFMAMBAF8w
DwYDVR0PAQH/BAUDAwcGADAdBgNVHQ4EFgQU00EGrJBt0UrrdaVKEJmzsaGLSwc
CgYIKoZIzj0EAwIDRwAwRAIgb+ZU2g6gWtKuEZ+Hxb/b/ad4lvvigtwjzRM4q3wgh
DdCIC0mA6AFwVwR91z4ZcyGbb0cNEjhAnFjXca4syc4XR7
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIICnCCaISgAwIBAgIMCL2F12yZJ6SAaEc7MAoGCCqGSM49BAMCMIGRMQswCQYD
VQGGewJVUzERMA8GA1UECBMISWxsaw5vaXNxEADA0BgNVBACTB0NoaWZhZ28xITAf
BgNVBAoTGFRydXN0d2F2ZSBib2xkaW5ncywgSW5jLjE6MDgGA1UEAxMVHJ1c3R3
YXZlIEdsb2JhbCBFQ0MgUDU1NiBDZXJ0aWZpY2F0aW9uIEF1dGhvcml0eTAeFw0x
NzA4MjMxOTM1MTBhFw00MjA4MjMxOTM1MTBhMIGRMQswCQYDVQGEwJVUzERMA8G
A1UECBMISWxsaw5vaXNxEADA0BgNVBACTB0NoaWZhZ28xITAfBgNVBAoTGFRydXN0
d2F2ZSBib2xkaW5ncywgSW5jLjE6MDgGA1UEAxMVHJ1c3R3YXZlIEdsb2JhbCBF
Q0MgUDU1NiBDZXJ0aWZpY2F0aW9uIEF1dGhvcml0eTBZMBMGByqGSM49AgEGSsB
BAAIA2IABGvaDXU1CDFHBA5FmVxXERMUSvgQMS0jfoPTfygIOiYa0s+Xgh+AtycJ
j9GOMMQmKmw6sWASr9zZ91C0kmwqKi6vr/Tk1ZvFe/oyuJUF5nQlgiip04pt892F
1PKYhDh1oKNDMEeWdWYDVR0TAQH/BAUwAwEB/zAPBgNVHQ8BAf8EBQMDbWYAMB0G
A1UdDgQwBBRvQYSJ0sEYvRjLbKYHTsjnnb6CKDAKBggqhkJOPQDQAwNnADBKAJA3
AZKXRRj+oPm+rRk6ct30UJMDer5E0k9BpIycnR+j9sKS50gU/k6bpZFXrsY3crsC
M0c1CrEMXU6pY5JvZAL/mYiykf9ijH3g/56vxC+Gcsej/YpPhRZ744hN8trMVKU
Sw==
-----END CERTIFICATE-----
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % grep -o -i "BEGIN CERTIFICATE" ca-certificates.crt | wc -l
127
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure %
```

- 127 certificates.

8. Extract a root certificate from ca-certificates.crt. Use the openssl command to explore the details. Do you see any Issuer information? Please compare it to the details of twitter's certificate and the details of the intermediate certificate.

```
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl x509 -in ca-certificates.crt -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 6828503384748696800 (0x5ec3b7a6437fa4e0)
        Signature Algorithm: sha1WithRSAEncryption
        Issuer: CN=ACCVRAIZ1, OU=PKIACCV, O=ACCV, C=ES
        Validity
            Not Before: May  5 09:37:37 2011 GMT
            Not After : Dec 31 09:37:37 2030 GMT
        Subject: CN=ACCVRAIZ1, OU=PKIACCV, O=ACCV, C=ES
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (4096 bit)
            Modulus:
                00:9b:a9:ab:bf:61:4a:97:af:2f:97:66:9a:74:5f:
                d0:d9:96:fd:cf:e2:e4:66:ef:1f:1f:47:33:c2:44:
                a3:df:9a:de:1f:b5:54:dd:15:7c:69:35:11:6f:bb:
                c8:0c:8e:6a:18:1e:d8:8f:d9:16:bc:10:48:36:5c:
                40:70:b3:00:f3:51:0f:07:d7:10:0f:15:00:74:50:
```

- From the image, the Issuer and Subject for Root CA are the same entity. This is called “Self-Signed Certificate”.

9. If the intermediate certificate is not in a PEM format (text readable), use the command to convert a DER file (.crt .cer .der) to PEM file.
- openssl x509 -inform der -in certificate.cer -out certificate.pem.**
- (You need the pem file for exercise 10.)

```
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure -- zsh -- 140x39
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl x509 -inform cer -in intermediate.cer -out intermediate.pem
bad input format specified for Certificate
unable to load certificate
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl x509 -inform der -in intermediate.cer -out intermediate.pem
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % ls
Activity_V-Public_Key_Infrastructure(Solution).docx  intermediate.pem
ca-certificates.crt                                twitter_com.cert
intermediate.cer                                   ~$tivity_V-Public_Key_Infrastructure(Solution).docx
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure %
```

10. From the given python code, implement the certificate validation.

```
1  from OpenSSL import crypto
2  import pem
3  import sys
4
5  intermediate_path = "./intermediate.pem"
6  ca_certificate_path = "./ca-certificates.crt"
7
8
9  def verify(filepath):
10     with open(filepath, "r") as cert_file:
11         cert = cert_file.read()
12
13     pems = pem.parse_file(ca_certificate_path)
14     trusted_certs = []
15     for mypem in pems:
16         trusted_certs.append(str(mypem))
17
18     int_pems = pem.parse_file(intermediate_path)
19     for mypem in int_pems:
20         trusted_certs.append(str(mypem))
21
22     verified = verify_chain_of_trust(cert, trusted_certs)
23
24     if verified:
25         print("Certificate verified")
26
27
28  def verify_chain_of_trust(cert_pem, trusted_cert_pems):
29     certificate = crypto.load_certificate(crypto.FILETYPE_PEM, cert_pem)
30
31     # Create and fill a X509Store with trusted certs
32     store = crypto.X509Store()
33
34     for trusted_cert_pem in trusted_cert_pems:
35         trusted_cert = crypto.load_certificate(crypto.FILETYPE_PEM, trusted_cert_pem)
36         store.add_cert(trusted_cert)
37
38     # Create a X509StoreContext with the cert and trusted certs
39     # and verify the the chain of trust
40     store_ctx = crypto.X509StoreContext(store, certificate)
41     # Returns None if certificate can be validated
42     result = store_ctx.verify_certificate()
43
44     return result is None
45
46
47  if __name__ == "__main__":
48     verify(filepath=sys.argv[1])
49
```


Twitter, google, www.chula.ac.th, classdeedee.cloud.cp.eng.chula.ac.th

```

Activity_V-Public_Key_Infrastructure -- openssl s_client -connect twitter.com -- 140x39

shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl s_client -connect twitter.com:443
CONNECTED(00000000)
depth=2 C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root CA
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert TLS Hybrid ECC SHA384 2020 CA1
verify return:1
depth=0 C = US, ST = California, L = San Francisco, O = "Twitter, Inc.", CN = twitter.com
verify return:1

Certificate chain
 0 s:/C=US/ST=California/L=San Francisco/O=Twitter, Inc./CN=twitter.com
 i:/C=US/O=DigiCert Inc/CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
 1 s:/C=US/O=DigiCert Inc/CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
 i:/C=US/O=DigiCert Inc/OU=www.digicert.com/CN=DigiCert Global Root CA

Server certificate
-----BEGIN CERTIFICATE-----
MIIFcTCCBPagAwIBAgIQBhwg1EwMtJeHzb2E7I4udjAKBgqhkJOPQQAABWBMQsw
CQYDVQQGEwJVUzEVMBMGA1UEChMMRGRlbnRlcnaUNlcnQSW5jMTAwLGYyDQVQDQYdEaWdp
Q2VydBUTFMGShllcmk1IEVDQyBTSEezODQGMjAyMCBDQTUWHhcNMjIwMjAyMDAw
MDAwHhcNMjIwMjAxMjI0TU5wJmBoMQswCQYDVQQGEwJVUzETMBEGA1UECBMKQ2Fz
aWZvcms5PjTEWMBQGA1UEBxMNNUZFuIEZyYW5jaXNjb2EWMBQGA1UECHMNVHdpdHRIRI
...
-----END CERTIFICATE-----

Server Temp Key: ECDH, X25519, 253 bits

SSL handshake has read 2918 bytes and written 289 bytes

New, TLSv1/SSLv3, Cipher is ECDHE-ECDSA-AES128-GCM-SHA256
Server public key is 256 bit
Secure Renegotiation IS supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol : TLSv1.2
    Cipher   : ECDHE-ECDSA-AES128-GCM-SHA256
    Session-ID: 37461564ABA234DCB2AE0A3C1BE86BE69CF664CED6437B6588CCE9857750D6E
    Session-ID-ctx:
    Master-Key: A3EBAC384CCF31DBE386DC2B9C6F26624BFCC102CEB0F02A92130E46D0A8289748ABE2A17A4162EB744E73C6F120741
    TLS session ticket lifetime hint: 7199 (seconds)
    TLS session ticket:
0000 - f5 73 ec c5 94 ff 54 6b-e2 60 98 42 90 26 d6 c6      .s....Tk.`.B.&m.
0010 - cb c0 d6 be 13 fb 23 b9-6e 2e 9a 7f b1 57 de 41      .....#.n....W.A
0020 - 1f 91 ba eb de be b1 1b-e5 ea 08 e3 0c 32 d7 dc      .....2..
0030 - 12 f7 39 b3 aa cf 0f fb-82 9a 9b b5 ca 98 1c ff      ..9.....
0040 - f5 33 cb 0c ff 42 4a d0-30 e1 77 07 6a e4 04 96      .3...BJ.O.w.j...
0050 - 77 b1 df 10 39 a3 4b 15-41 fd 2d b5 47 2c db 7a      wQ..9.K.A.-.G,.z
0060 - 47 4c 7f 8c de d6 d0 94-cb a1 7c e5 85 7b b9 c1      GL.....|.f..
0070 - 22 f7 75 38 25 38 cd 12-ed f0 89 c7 cb 67 2c da      ".u8%8.....g..
0080 - 04 aa 04 e6 51 e9 2f 04-c4 cf 50 2d 6a 88 9d 09      ....Q./...P-j...
0090 - 74 b0 4c 45 e9 08 06 44-68 ca b1 39 9d 73 9d 85      t.LE...Dh..9.s..
00a0 - 7c de 4a 70 f3 d1 ac 05-b8 68 21 9b 8e 84 bf 61      |.Jp.....h!...a
00b0 - 4a b1 4d dc cf cb 3c b5 d9-61 3f 2f 53 c8 17 49 06      J.M...<..a?S.I.

Start Time: 1663054069
Timeout : 7200 (sec)
Verify return code: 0 (ok)

^C
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % python app.py twitter_com.cert
Certificate verified

```

- Google (verified)

```
Activity_V-Public_Key_Infrastructure -- zsh -- 140x39
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl s_client -connect google.com:443
CONNECTED(00000006)
depth=3 C = BE, O = GlobalSign nv-sa, OU = Root CA, CN = GlobalSign Root CA
verify return:1
depth=2 C = US, O = Google Trust Services LLC, CN = GTS Root R1
verify return:1
depth=1 C = US, O = Google Trust Services LLC, CN = GTS CA 1C3
verify return:1
depth=0 CN = *.google.com
verify return:1
---
Certificate chain
 0 s:/CN=*.google.com
  i:/C=US/O=Google Trust Services LLC/CN=GTS CA 1C3
 1 s:/C=US/O=Google Trust Services LLC/CN=GTS CA 1C3
  i:/C=US/O=Google Trust Services LLC/CN=GTS Root R1
 2 s:/C=US/O=Google Trust Services LLC/CN=GTS Root R1
  i:/C=BE/O=GlobalSign nv-sa/OU=Root CA/CN=GlobalSign Root CA
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIOuCCDaCgAwIBAgIQcy6dHdF2sBwKkvTfhLZn/DANBgkqhkiG9w0BAQsFADBG
MQswCQYDVQQGEwJVUzEiMCAGA1UEChMZR29vZ2x1IFRydXN0IFN1cnZpY2VzIExm
QzETMBEGA1UEAxMKR1RTIENBIDFDMzAeFw0yMjA4MjIwODIxNTVaFw0yMjExMTQw
ODIxNTRaMBcxFTATBgNVBAMMDCouZ29vZ2x1LmNvbTCCASIwDQYJKoZIhvcNAQEB
-----
```

```
Activity_V-Public_Key_Infrastructure -- zsh -- 139x24

TLS session ticket:
0000 - 02 2c 13 b3 ba 62 e7 e4-1a f8 b6 14 1c b6 b4 9e  ,...b.....
0010 - 39 7a 68 c4 17 6d d8 a8-b0 11 b2 63 bd da ce db  9zh..m....c....
0020 - 17 27 17 7b 1a 73 83 50-54 55 83 81 45 d4 59 06  .'.{.s.PTU..E.Y.
0030 - 47 9d 0e 31 51 e8 f1 34-28 05 af e0 34 4e 9c d8  G..1Q..4(...4N..
0040 - 3a 0f 71 8a ad 43 58 7d-b9 22 41 e5 20 c1 aa df  :.q..CX).A. ...
0050 - a6 13 ab 99 7c 47 ec 91-eb fc 47 99 c3 b0 bf 10  ....|G....G....
0060 - c9 ce dd 2e 58 b4 b0 12-cb 49 83 07 b5 6e b3 af  ....X....I...n..
0070 - 03 fe 90 8a ec 86 d7 de-28 9a cb 8a 37 b2 24 98  ....7.$.....
0080 - 14 0f 85 4a 54 ed f3 83-9b 51 fc 55 35 88 71 c6  ...JT....Q.U5.q.
0090 - 53 f6 7f 10 98 17 38 3d-78 9a 8c 7b 85 98 dd bc  S.....8=x.{...
00a0 - da 01 38 f2 20 cf 87 32-84 6c e0 18 44 84 f8 ef  ..8. .2.l..D...
00b0 - 0e d9 b1 d3 e5 87 3c fd-a3 3b 10 e1 19 e6 0a a8  ....<.;.....
00c0 - db b3 80 04 e3 4a 24 a3-93 1d 90 19 cf 92 62 12  ....JS$.....b.
00d0 - 23 2c 6d 15 39 53 49 4e-bb 7e 92 29 2d bc 7b 8e  #,m.9SIN.-.)-.{.

Start Time: 1663246372
Timeout : 7200 (sec)
Verify return code: 0 (ok)

^C
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % python app.py google.crt
Certificate verified
```

- Chula (verified)

```

shhria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl s_client -connect chula.ac.th:443
CONNECTED(00000006)
depth=2 OU = GlobalSign Root CA - R3, O = GlobalSign, CN = GlobalSign
verify return:1
depth=1 C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2022 Q3
verify return:1
depth=0 CN = imperva.com
verify return:1
---
Certificate chain
 0 s:/CN=imperva.com
  i:/C=BE/O=GlobalSign nv-sa/CN=GlobalSign Atlas R3 DV TLS CA 2022 Q3
 1 s:/C=BE/O=GlobalSign nv-sa/CN=GlobalSign Atlas R3 DV TLS CA 2022 Q3
  i:/OU=GlobalSign Root CA - R3/O=GlobalSign/CN=GlobalSign
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIGbzCBGwAwIBAgIQAAQv5hzLlYJi5h6MIweUeRTANBgkqhkiG9w0BAQsFADBY
MQswCQYDVQQGEwJCRTEZMBcGA1UEChMQR2xvYmF1U2lnbiBud1izYTEuMCwGA1UE
AxMlR2xvYmF1U2lnbiBDbG9keycBSMyBEViBUTFmG0QEGMjAyMjAyMjAyMjA3

```

```

Server public key is 2048 bit
Secure Renegotiation IS supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol : TLSv1.2
    Cipher   : ECDHE-RSA-AES128-GCM-SHA256
    Session-ID: 968E41400D6F527A79858189B6D7CF30A92ADEE4A0A30B43665F6711A0E0FCC
    Session-ID-ctx:
    Master-Key: 7EE0761669D80DC4A01690D88FEE334FB576BB8A7A0404F9BAEA07A3539F09EFAF3778FC2DFA15027CBF83D5CA7BC7CE
    TLS session ticket lifetime hint: 7200 (seconds)
    TLS session ticket:
0000 - a4 35 8f 72 c5 c2 7e 95-d4 a8 f9 ec 88 a2 66 3a      .5.r.~.....f:
0010 - 0b 45 f3 54 41 db 9c dd-83 d9 1a 1b ed 0b b6 f5      .E.TA.....
0020 - b5 af db f1 f7 47 85 f9-34 41 29 5a bf dc f4 27      ....G..4A)Z...'
0030 - 2d a1 5b 54 d5 33 63 7c-95 cd fd a9 40 e2 a9 80      ~.[.3c]....@...
0040 - 90 ee 0c ef de 8d bf 4d-88 42 2e 5f 21 e4 93 bd      ....M.B._...
0050 - 37 fc be 19 31 3d 41 87-ab ce 03 04 1d 48 ae 70      7...1=A.....H.p
0060 - 76 8c 92 54 b0 a4 72 f2-ca 44 ca 2e 12 12 4d 24      v..T..r..D....MS
0070 - c5 ee 6a aa 93 d4 e8 45-1e e4 00 2b 24 67 9f 14      ..j....E...+Sg..
0080 - a7 7d 7c 2e ca 79 61 82-fb dd 04 46 e9 01 43 4e      .}|..ya.....F..CN
0090 - 40 d4 46 28 e5 40 6d 8b-c7 2b 88 34 88 6e 6b e7      @j{(.@m...+4.nk.

Start Time: 1663246533
Timeout : 7200 (sec)
Verify return code: 0 (ok)

^C
shxria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % python app.py chula.crt
Certificate verified

```

- Classdeedee (self-signed certificate)

```
Activity_V-Public_Key_Infrastructure -- -zsh -- 140x39
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % openssl s_client -connect classdeedee.cloud.cp.eng.chula.ac.th:443
CONNECTED(00000000)
depth=0 0 = Acme Co, CN = Kubernetes Ingress Controller Fake Certificate
verify error:num=20:unable to get local issuer certificate
verify return:1
depth=0 0 = Acme Co, CN = Kubernetes Ingress Controller Fake Certificate
verify error:num=21:unable to verify the first certificate
verify return:1
---
Certificate chain
 0 s:/O=Acme Co/CN=Kubernetes Ingress Controller Fake Certificate
 1 i:/O=Acme Co/CN=Kubernetes Ingress Controller Fake Certificate
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIDcDCCA1gAwIBAgIRAPv1Cw8yeZf25f7x2lfSNHgwDQYJKoZIhvcNAQELBQAw
SzEQMA4GA1UEChMHQWNTZSBDbzE3MDUGA1UEAxMuS3ViZXJlcyBJbmdyZXNz
IENvbnRyb2xsZXIwRmFrZSBDbzE3MDUGA1UEAxMuS3ViZXJlcyBJbmdyZXNz
MzA1MjAwNzQwNDdaMExsEDAOBgNVBAoTB0FjbWUgQ28xNzA1BgNVBAMTLkt1YmV5
-----
Master-Key: BCC4C8CA34A9C799F003F014735DF0B4C000266A96BC06C633293AB3452139DFC3BF6E1C4633BE1BB8E3CCB81109EC81
TLS session ticket lifetime hint: 600 (seconds)
TLS session ticket:
0000 - a0 9d 5b 14 4f 4e 2a 4a-71 b5 41 55 51 b0 a7 22 ..[.ON*Jq.AUQ..
0010 - 9f bc e9 53 05 56 23 67-c3 9c 60 86 8e 2c e3 29 ...S.V#g..`....)
0020 - 3a af df 90 f8 2a 78 b3-08 27 05 65 47 b7 ac 47 :....*x..'eG..G
0030 - 2e 21 64 ea a9 47 66 b6-08 7a 61 e2 2e e4 ad a9 .ld..Gff.za....
0040 - 94 62 3c 10 c0 f6 f4 90-fb b1 de 02 ea e7 7a fa .b<.....z.
0050 - 06 76 e3 c3 54 39 c6 bd-99 1b 50 e0 c6 bb 8e 8a .v..T9l...P.....
0060 - 5b 3f 6e 37 d0 29 95 08-48 ad 04 ea e8 40 4b 29 [?n7.)...H....@K)
0070 - 20 1b ad 6e 15 8d f4 69-00 3a 59 51 c8 ea 65 97 .n...i.:YQ..e.
0080 - b0 70 5a 3e 79 7a 64 5e-e6 0a b3 16 e0 5f 60 8f .pZ>yzd^.....`
0090 - ae b4 3c dc d5 5c f3 3d-1a 4b d8 7c 16 6d 51 71 ..<..\.=.K.|.mQq
00a0 - db b0 f8 46 c8 75 b4 48-d3 67 b5 17 9b 00 db d3 ...F.u.H.g.....

Start Time: 1663246657
Timeout : 7200 (sec)
Verify return code: 21 (unable to verify the first certificate)
---
^C
shrria@Suchakrees-MacBook-Air Activity_V-Public_Key_Infrastructure % python app.py classdeedee.crt
Traceback (most recent call last):
  File "app.py", line 48, in <module>
    verify(filepath=sys.argv[1])
  File "app.py", line 22, in verify
    verified = verify_chain_of_trust(cert, trusted_certs)
  File "app.py", line 42, in verify_chain_of_trust
    result = store_ctx.verify_certificate()
  File "/Users/shrria/Library/Python/3.8/lib/python/site-packages/OpenSSL/crypto.py", line 1896, in verify_certificate
    raise self._exception_from_context()
OpenSSL.crypto.X509StoreContextError: [18, 0, 'self-signed certificate']
```

11. Nowadays, there are root certificates for class 1 and class 3. What uses would a class 1 signed certificate have that a class 3 doesn't, and vice versa?

- Class 1 signed certificate is a low-level of certificate which allows the owner to self-verify their website with low-security authentication. The class-1 certificate is used for a personal or any website that does not require high level of security.
- Class 3 signed certificate is available for a subject who oversees sensitive information such as banking transaction requests, or any confidential data sent over network.

12. Assuming that a Root CA in your root store is hacked and under the control of an attacker, and this is not noticed by anyone for months.

a) What further attacks can the attacker stage? Draw a possible attack setup.

- When your computer considered the fake CA trustworthy, you could get attacked by the hackers with other tricks leading you to enter the injected-scripted website or download harmful things from their self-signed website that look like a genuine one.

b) In the attack you have described above, can we rely on CRLs or OCSP for protection? Please explain

- No.
- CRL (Certificate Revocation List) is the list of certificates which are expired or changed a status to unauthorized from a single CA.
- OCSP (Online Certificate Status Protocol) is a protocol to acknowledge the status of certificates which issued by a single CA as well.
- If the attacker has added a Root CA into the root store, both CRL and OCSP were in control of the attacker as well. Because the Root CA is the organization who individually announces the CRL and allows accessibility to view OCSP.
- To avoid such an attack, a user must remove a Root CA of an attacker from the device and only remain trustful Root CAs and their intermediates.